

5.8 Dual Bluetooth ESP32

Dual ESP32 BLE Bluetooth Communication

What You Will Build

After completing this project, you will have a **bidirectional Bluetooth communication system** between **two ESP32 development boards**:

- ▶  **ESP32 Slave**: Acts as a BLE peripheral, advertising and waiting for connection
- ▶  **ESP32 Master**: Acts as a BLE central device, scanning for and connecting to the Slave
- ▶  **Bidirectional Chat**:
 - ▶ Master serial input → Slave serial display
 - ▶ Slave serial input → Master serial display
- ▶  **LED Command Control**: Enter `/ledon` or `/ledoff` on either side to control **only the remote** board's onboard LED
- ▶  **Dual Board LED**: Both boards' GPIO2 LEDs can be **remotely** controlled (entering a command controls only the remote board, not the local one)

Simple analogy: Like two people using walkie-talkies — the Master is the one who initiates the call, the Slave is the one who receives it. Both can talk and listen, and both can remotely control the other's device.



 **Difference from single-board + web version**: The web version requires a computer browser as the central device; this project uses two ESP32 boards communicating directly, no computer needed (except for flashing and serial debugging), making it closer to real IoT device-to-device communication.

Step 1: Preparation (Hardware + Software)

Hardware List

Device	Description
ESP32 dev board × 2	Must support BLE (ESP32 / ESP32-WROOM / ESP32-S3 all work)
USB data cable × 2	For flashing code and serial communication
Computer × 1	For flashing code and serial debugging (can connect both boards simultaneously)

Software List

Software	Description
Arduino IDE	For compiling and flashing ESP32 code
Serial terminal (optional)	e.g., PuTTY, SSCOM, etc. — convenient for viewing both boards' output simultaneously

Communication Flow

1. **Slave starts**: Initializes BLE, begins advertising as `ESP32_BLE_Slave`
2. **Master starts**: Initializes BLE, scans for nearby BLE devices
3. **Master discovers Slave**: Finds the Slave device with the matching service UUID
4. **GATT connection established**: Master connects to the Slave's BLE service
5. **Bidirectional chat**:
 - ▶ Master serial input → Written to Slave via BLE → Displayed on Slave serial
 - ▶ Slave serial input → Notified to Master via BLE → Displayed on Master serial
6. **LED control** (controls the remote, not local):
 - ▶ Master enters `/ledon` → Sends command to Slave → **Only Slave LED turns on** (Master LED unchanged)
 - ▶ Slave enters `/ledoff` → Sends command to Master → **Only Master LED turns off** (Slave LED unchanged)
7. **Disconnect & reconnect**: Automatically attempts to reconnect after disconnection

Step 2: Software Configuration & Flashing

[Back to Elegoo](#)

 [DOWNLOAD](#)

Step 2.1: Open the Project Code

1. Place the entire project folder somewhere on your computer you can find
2. You will see the following structure:
 - [ESP32_Slave/ESP32_Slave.ino](#) → Slave code (advertises and waits for connection)
 - [ESP32_Master/ESP32_Master.ino](#) → Master code (scans and connects)

Step 2.2: Modify Configuration (Optional)



 **The default configuration works out of the box!** If you want to customize the Bluetooth name or UUID, just modify the corresponding code.

Change Bluetooth name (in both .ino files):

```
1 | // In Slave:
2 | BLEDevice::init("ESP32_BLE_Slave"); // Change to your desired slave name
3 |
4 | // In Master:
5 | BLEDevice::init("ESP32_BLE_Master"); // Change to your desired master name
```



 If you change the Slave's advertising name, the Master code does not need to be modified (Master identifies devices by service UUID, not by name).

Change service UUID (must be identical in both .ino files):

```
1 | #define SERVICE_UUID "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
```



 The UUID is the BLE device's ID card. If you change the UUID in one file, the other must be updated as well, otherwise communication will fail.

Step 2.3: Flash ESP32 Code

Important: Each board gets different code!

ESP32 Board	Code to Flash	Role
Board A	ESP32_Slave.ino	Slave — advertises and waits for connection
Board B	ESP32_Master.ino	Master — scans and connects

Flashing steps:

1. Connect the first ESP32 (Slave) to your computer via USB
2. Open [ESP32_Slave.ino](#) in Arduino IDE
3. Select the correct board and port:
 - **Tools** → **Board** → Select [ESP32 Dev Module](#)
 - **Tools** → **Port** → Select the corresponding COM port
4. Click the "Upload" button (or press **Ctrl+U**) to flash the code
5. After flashing is complete, connect the second ESP32 (Master) to your computer
6. Open [ESP32_Master.ino](#) in Arduino IDE
7. Select the correct board and port (may be a new COM port)
8. Click "Upload" to flash the code



 If your computer has only one USB port, you can flash the boards one at a time. If you have two USB ports, you can connect and flash both simultaneously.

Step 2.4: Open the Serial Monitor

[Back to Elegoo](#)

1. Open the Serial Monitor in Arduino IDE ([Ctrl+Shift+M](#))
2. Set the **baud rate** to **115200** (very important! Otherwise you'll see garbled text)
3. ⚠ **Critical setting:** Set the **line ending** to **"Newline"** or **"Both NL & CR"**
4. Press the **EN button** (reset button) on the ESP32
5. Observe the serial output and compare it with the "Expected Output" in the next section



It's recommended to use two serial monitor windows to view each board's output separately. Arduino IDE only supports one serial monitor by default, so you can use PuTTY or SSCOM to monitor two COM ports simultaneously.

✅ Step 3: Verify the Results

3.1 Slave Serial Expected Output

```
1 | =====
2 | ESP32 BLE Slave — Starting up...
3 | =====
4 | [Slave] BLE device name: ESP32_BLE_Slave
5 | [Slave] Advertising started. Waiting for Master connection...
6 |
7 | [Slave] Type a message and press Enter to send to Master.
8 | [Slave] Commands: /ledon /ledoff /help
```

- ▶ See [BLE device name: ESP32_BLE_Slave](#) → BLE initialization successful ✓
- ▶ See [Advertising started](#) → Advertising started, waiting for Master to connect ✓

3.2 Master Serial Expected Output

```
1 | =====
2 | ESP32 BLE Master — Starting up...
3 | =====
4 | [Master] Scanning for ESP32_BLE_Slave...
5 | [Master] Found target device: ESP32_BLE_Slave
6 | [Master] Connecting to ESP32_BLE_Slave...
7 | [Master] >>> Connected to Slave!
8 | [Master] Connected! Discovering services...
9 | [Master] Service found! Discovering characteristics...
10 | [Master] Subscribed to Slave notifications.
11 | [Master] Ready! Type a message and press Enter to send.
```

- ▶ See [Found target device](#) → Scan successful, Slave discovered ✓
- ▶ See [Connected to Slave!](#) → Connection established ✓
- ▶ See [Subscribed to Slave notifications](#) → Notification subscription successful, ready to chat ✓

Meanwhile, the Slave serial will show:

```
1 | [Slave] >>> Master connected!
```

3.3 Bidirectional Chat

Type a message in the Master serial monitor, press Enter:

- ▶ Master serial shows: [\[Master\] Sent: Hello Slave!](#)
- ▶ Slave serial shows: [\[Slave\] Received: Hello Slave!](#)

Type a message in the Slave serial monitor, press Enter:

- ▶ Slave serial shows: [\[Slave\] Sent: Hello Master!](#)
- ▶ Master serial shows: [\[Master\] Received: Hello Master!](#)

3.4 LED Command Control



Rule: Entering a command controls only the remote board's LED, not the local board's LED.

Master enters `/ledon` (controls Slave LED):

[Back to Elegoo](#)

Master (sender):

```
1 | [Master] >>> Sent command: turn Slave LED ON
```

Master LED **unchanged** ✓

Slave (receiver):

```
1 | [Slave] >>> Master command: LED turned ON
```

Slave onboard LED (GPIO2) turns on ✓

Slave enters `/ledon` (controls Master LED):

Slave (sender):

```
1 | [Slave] >>> Sent command: turn Master LED ON
```

Slave LED **unchanged** ✓

Master (receiver):

```
1 | [Master] >>> Slave command: LED turned ON
```

Master onboard LED (GPIO2) turns on ✓

`/ledoff` works the same way: Whichever side you enter it on, it turns off the **remote** board's LED.

View help: Enter `/help` on either side, the remote side will display the available command list.

3.5 Disconnect & Reconnect

If the connection between the two boards is lost:

Master serial:

```
1 | [Master] <<< Disconnected from Slave. Reconnecting...
2 | [Master] Connecting to ESP32_BLE_Slave...
3 | [Master] >>> Connected to Slave!
```

Slave serial:

```
1 | [Slave] <<< Master disconnected. Restarting advertising...
2 | [Slave] >>> Master connected!
```

Both boards will automatically attempt to reconnect — no manual intervention required.

🤔 Step 4: Troubleshooting



💡 **New users start here:** 90% of issues are covered below.

Q1: Serial monitor shows garbled text or blank?

A: Baud rate is incorrect.

- Check the baud rate in the bottom-right of the serial monitor — **must be 115200**
- After changing, press the EN (reset) button on the ESP32

Q2: No response after typing in serial monitor?

A: Line ending setting is incorrect.

[Back to Elegoo](#)

- ▶ Set the serial monitor's line ending to "**Newline**" or "**Both NL & CR**"
- ▶ Must NOT be "No line ending"

Q3: Compilation error — BLE library not found?

A: ESP32 core version is too old or wrong board selected.

1. Update the [esp32](#) core to the latest version in Arduino IDE's "Boards Manager"
2. **Tools** → **Board** → Select [ESP32 Dev Module](#) (do not select ESP32-S2)
3. The BLE library is included with the ESP32 core — no additional installation needed

Q4: Master can't find the Slave device?

A: Slave is not advertising or is too far away.

1. **Verify Slave is flashed and running:** Serial should show [Advertising started](#)
2. **Press EN button to restart Slave:** Sometimes the advertising state gets stuck, a restart fixes it
3. **Move closer:** Place both ESP32 boards within 1 meter
4. **Check service UUID:** Confirm [SERVICE_UUID](#) is identical in both code files

Q5: Master connection fails?

A: Multiple possible causes.

1. **Verify Slave is still advertising:** Slave serial should show [Advertising started](#)
2. **Verify no other device is connected to Slave:** BLE peripherals typically support only one connection
3. **Restart both boards:** Sometimes the BLE protocol stack gets into a bad state
4. **Check UUID matching:** Service UUID and Characteristic UUID must be identical

Q6: Connected but can't receive messages?

A: Characteristic subscription may have failed or UUIDs don't match.

1. **Verify Master serial shows:** Should have [Subscribed to Slave notifications](#)
2. **Verify Slave serial shows connection:** Should have [>>> Master connected!](#)
3. **Check RX/TX UUID:** [CHARACTERISTIC_UUID_RX](#) and [CHARACTERISTIC_UUID_TX](#) must be identical in both code files
4. **Restart both boards and retry**

Q7: LED doesn't light up?

A: Check the following:

1. **Verify the command was sent:** Sender's serial should show [LED turned ON](#)
2. **Verify LED pin:** Code uses [LED_PIN = 2](#) (GPIO2), most ESP32 boards have onboard LED on this pin
3. **Some boards use active-low LED:** Some ESP32 boards' LEDs are active-low, requiring [HIGH](#) and [LOW](#) to be swapped
4. **Observe receiver's serial:** Should show the corresponding command log

Q8: Only one side receives messages?

A: One side's characteristic may have an issue.

1. **Check Master's TX subscription:** Master must successfully subscribe to TX characteristic notifications to receive Slave messages
2. **Check Slave's TX notification:** Slave's TX characteristic must have the [BLE2902](#) descriptor added
3. **Verify bidirectional UUID consistency:** RX and TX UUIDs must match in both files

Q9: Long messages are incomplete?

A: BLE default MTU limit.

- ▶ The code sets [BLEDevice::setMTU\(512\)](#) , allowing single messages up to ~500 bytes
- ▶ If messages are longer, they need to be sent in segments

► Normal chat messages won't exceed this limit

[Back to Elegoo](#)

Q10: Code changes don't take effect?

- A:
- 1. You must re-flash the corresponding board after modifying code
 - 2. When both sides' UUIDs are modified, both boards must be re-flashed
 - 3. After changing the Bluetooth name, the Master side needs no changes (identifies via UUID)

🔍 Quick Troubleshooting Table

Symptom	Most Likely Cause	Fastest Solution
Garbled / blank serial	Wrong baud rate	Change to 115200, press EN to reset
No response to serial input	Wrong line ending	Set to "Newline" or "Both NL & CR"
Compile error — BLE library not found	ESP32 core too old	Update esp32 core to latest
Master can't find Slave	Slave not advertising / too far	Restart Slave, move boards closer
Master connection fails	UUID mismatch / already connected	Check UUIDs, restart both boards
Connected but no messages	Characteristic mismatch / subscription failed	Check UUIDs, restart and reconnect
LED doesn't light up	Wrong pin / active-low LED	Confirm GPIO2, try swapping HIGH/LOW
Only one-way communication	TX subscription failed	Check BLE2902 descriptor and notify callback
Changes don't take effect	Not re-flashed	Re-upload code to the corresponding board

📖 Step 5: Understanding the Project Principles

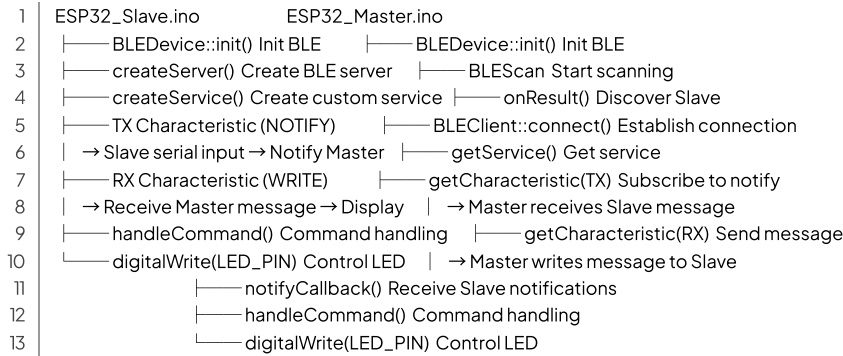
💡 After completing the experiment, understanding the principles behind it will be much easier!

5.1 Project Architecture: Master / Slave Division

This project uses a **dual-board** architecture:

Device	Role	Main Responsibilities	Analogy
ESP32 Slave	BLE Peripheral	Advertise Bluetooth, wait for connection, provide services, respond to commands	Receiver / Controlled end
ESP32 Master	BLE Central	Scan devices, establish connection, send requests, actively control	Initiator / Controller

Module relationship diagram:



5.2 BLE GATT Protocol Overview

BLE uses the **GATT (Generic Attribute Profile)** protocol for data exchange. The core concepts are "services" and "characteristics":

Concept	Analogy	Description
Service	An app	A collection of related functions, identified by UUID
Characteristic	A data item in an app	The actual data-carrying unit, with different properties

The custom service in this project contains two characteristics:

Characteristic	UUID	Property	Direction
RX	beb5483e-...	Write	Master → Slave
TX	8d8f28a9-...	Notify	Slave → Master



Note: `writeValue()` requires `uint8_t*` type. When using `c_str()`, a cast is needed: `writeValue((uint8_t*)str.c_str(), len, false)`. The third parameter set to `false` means Write Command (no response), matching the Slave's `PROPERTY_WRITE_NR` property. Alternatively, use the `String` overload: `writeValue(packet, false)`.

5.3 BLE Communication Modes: Peripheral vs Central

BLE communication involves two main roles:

Role	Description	Analogy
Peripheral	Broadcasts itself, waits for other devices to connect	Bluetooth speaker, smartwatch
Central	Scans for and connects to other devices	Phone, computer

In this project:

- ▶ **Slave** acts as Peripheral: Advertises as `ESP32_BLE_Slave`, waits for connection
- ▶ **Master** acts as Central: Scans for and connects to the Slave



Note: This is similar to the "master/slave" concept in Classic Bluetooth, but BLE's Peripheral/Central roles can be swapped during connection (Role Switching). In this project, the Master is the initiator and the Slave is the passive party.

5.4 BLE Connection Establishment Flow

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13
- 14
- 15
- 16
- 17
- 18
- 19
- 20
- 21
- 22
- 23
- 24
- 25
- 26
- 27
- 28
- 1. Slave: `BLEDevice::init("ESP32_BLE_Slave")`
 - ↳ Initialize BLE protocol stack
 - 2. Slave: `createService() + createCharacteristic()`
 - ↳ Create GATT service and characteristics
 - 3. Slave: `BLEDevice::startAdvertising()`
 - ↳ Start advertising, wait for connection
 - 4. Master: `BLEScan::start()`
 - ↳ Scan for nearby BLE devices
 - 5. Master: `onResult()` callback
 - ↳ Discover Slave with matching Service UUID
 - 6. Master: `BLEClient::connect(targetDevice)`
 - ↳ Initiate GATT connection
 - 7. Slave: `onConnect()` callback
 - ↳ Accept connection, stop advertising
 - 8. Master: `getService() → getCharacteristic()`
 - ↳ Discover service and characteristics
 - 9. Master: `registerForNotify()`
 - ↳ Subscribe to Slave's TX notifications
 - 10. Bidirectional communication established!

5.5 Why BLE Instead of Classic Bluetooth SPP?

[Back to Elegoo](#)

Feature	Classic BT (SPP)	BLE	This Project
Transport model	Serial stream	GATT service/characteristic	GATT
Power consumption	Higher	Very low	BLE
Connection speed	Slower (pairing)	Fast (direct connection)	BLE
Reliability	Stream-based	Packet-based	BLE
Chip support	ESP32 (original) only	ESP32 / S3 / C3, etc.	BLE
Multi-connection	Usually 1	Multiple (mode dependent)	BLE

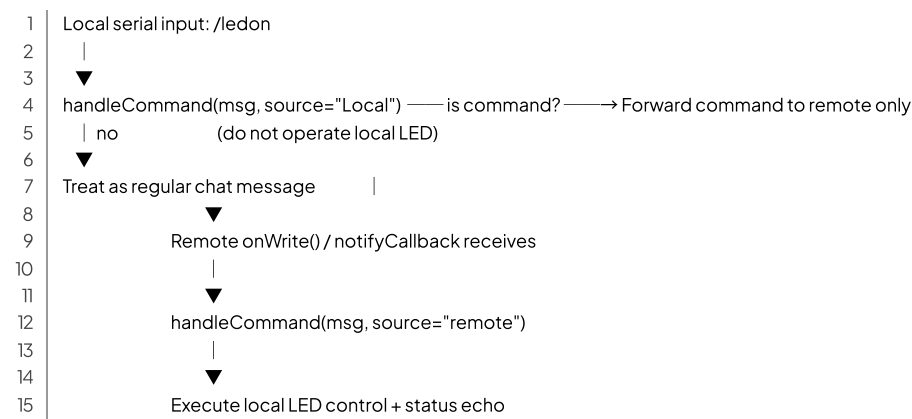
Why choose BLE: BLE is low-power Bluetooth with fast connection speed, low power consumption, and a broader ecosystem. Although Classic Bluetooth SPP can also achieve dual-board communication, BLE is more aligned with IoT trends and can directly interconnect with phones, web pages, and other BLE devices.

5.6 Command System Design

This project uses the `/` prefix to distinguish commands from regular messages. Both sides share the same command protocol:

Command	Sender	Effect
<code>/ledon</code>	Master or Slave	Turn on the remote board's GPIO2 LED (local board unchanged)
<code>/ledoff</code>	Master or Slave	Turn off the remote board's GPIO2 LED (local board unchanged)
<code>/help</code>	Master or Slave	Request available command list from the remote side

Command processing flow (using `/ledon` as an example):



This "command + regular message" coexistence design is identical to chat bots and smart home control protocols — the same channel can carry both data and control commands. The `source` parameter distinguishes "local input" from "remote command", naturally implementing the "control remote only" semantics.

5.7 BLE Callback Mechanism

BLE uses **callback functions** to handle asynchronous events:

Slave-side callbacks:

Callback	Trigger	Action
<code>ServerCallbacks::onConnect()</code>	Master connects	Set <code>deviceConnected = true</code>
<code>ServerCallbacks::onDisconnect()</code>	Master disconnects	Set <code>deviceConnected = false</code> , restart advertising

Callback	Trigger	Action
<code>RxCallbacks::onWrite()</code>	Master sends data	Parse message, handle command or display chat

Master-side callbacks:

Callback	Trigger	Action
<code>ClientCallbacks::onConnect()</code>	Connected to Slave	Set <code>deviceConnected = true</code>
<code>ClientCallbacks::onDisconnect()</code>	Connection lost	Set <code>deviceConnected = false</code> , reconnect
<code>ScanCallbacks::onResult()</code>	Device found during scan	Check if UUID matches
<code>notifyCallback()</code>	Slave sends notification	Parse message, handle command or display chat



 **Callbacks vs Polling:** Callbacks are "event-driven" — they execute only when something happens, not wasting CPU when idle. This is much more efficient than constantly checking in `loop()`.

Step 6: Code Walkthrough

6.1 Libraries and Key Functions

6.1.1 Slave-side (Server) Libraries

Library	Description
<code>BLEDevice.h</code>	BLE device management, initialization, creating server
<code>BLEServer.h</code>	BLE server, handling connect/disconnect callbacks
<code>BLEUtils.h</code>	BLE utility classes
<code>BLE2902.h</code>	CCC descriptor, required for notify functionality

6.1.2 Master-side (Client) Libraries

Library	Description
<code>BLEDevice.h</code>	BLE device management, initialization
<code>BLEScan.h</code>	BLE scanning, device discovery
<code>BLEClient.h</code>	BLE client, connecting to remote devices
<code>BLERemoteCharacteristic.h</code>	Remote characteristic, reading/writing data
<code>BLEUtils.h</code>	BLE utility classes
<code>BLE2902.h</code>	CCC descriptor, required for notify functionality

6.1.3 BLE Key Function Reference

Function	Side	Description
<code>BLEDevice::init(name)</code>	Both	Initialize BLE device, set name
<code>BLEDevice::setMTU(512)</code>	Both	Set MTU, affects max transfer bytes
<code>BLEDevice::createServer()</code>	Slave	Create BLE server
<code>BLEDevice::getScan()</code>	Master	Get BLE scanner

Function	Side	Description
<code>pServer->createService(uuid)</code>	Slave	Create BLE service
<code>pService->createCharacteristic(uuid, props)</code>	Slave	Create characteristic
<code>pTx->addDescriptor(new BLE2902())</code>	Slave	Add CCC descriptor (required for notify)
<code>pAdvertising->start()</code>	Slave	Start advertising
<code>pScan->start(0, false)</code>	Master	Start scanning (0 = unlimited)
<code>pClient->connect(device)</code>	Master	Connect to remote device
<code>pClient->getService(uuid)</code>	Master	Get remote service
<code>pService->getCharacteristic(uuid)</code>	Master	Get remote characteristic
<code>pTx->registerForNotify(callback)</code>	Master	Register notification callback
<code>pRx->writeValue(packet, false)</code>	Master	Write data to remote characteristic, <code>false</code> = Write Command (no response), matching Slave's <code>WRITE_NR</code> property

6.2 Slave Code Walkthrough: ESP32_Slave.ino

6.2.1 Configuration Section

```

1 | #define SERVICE_UUID    "4fafc201-1fb5-459e-8fcc-c5c9c331914b"
2 | #define CHARACTERISTIC_UUID_RX "beb5483e-36e1-4688-b7f5-ea07361b26a8"
3 | #define CHARACTERISTIC_UUID_TX "8d8f28a9-467b-4f25-a019-7cc5e6c8f721"
4 |
5 | #define LED_PIN 2
6 |
7 | BLEServer *pServer = nullptr;
8 | BLECharacteristic *pTxCharacteristic = nullptr;
9 | bool deviceConnected = false;

```

Logic: Defines BLE service UUID, characteristic UUIDs, LED pin, and state variables.

Key points:

- ▶ The three UUIDs **must be identical in both .ino files**
- ▶ `RX` characteristic: Master writes data here (Master → Slave)
- ▶ `TX` characteristic: Slave sends data via notification (Slave → Master)
- ▶ `deviceConnected` : Connection state flag

6.2.2 sendViaBLE(): Send Message to Master

```

1 | void sendViaBLE(const String &msg) {
2 |     if (deviceConnected && pTxCharacteristic != nullptr) {
3 |         pTxCharacteristic->setValue((msg + "\n").c_str());
4 |         pTxCharacteristic->notify();
5 |         delay(10);
6 |     }
7 | }

```

Logic: Checks connection state, writes message to TX characteristic and sends notification.

Key points:

- ▶ `setValue((msg + "\n").c_str())` : Sets the data to send, appends `\n` as message delimiter
- ▶ `notify()` : Actively pushes data to the subscribed Master
- ▶ `delay(10)` : Gives the BLE protocol stack time to send data

6.2.3 BLE Callback Functions

```

1  class ServerCallbacks : public BLEServerCallbacks {
2      void onConnect(BLEServer *pServer) {
3          deviceConnected = true;
4      }
5      void onDisconnect(BLEServer *pServer) {
6          deviceConnected = false;
7          BLEDevice::startAdvertising();
8      }
9  };
10
11 class RxCallbacks : public BLECharacteristicCallbacks {
12     void onWrite(BLECharacteristic *pCharacteristic) {
13         String msg = pCharacteristic->getValue();
14         msg.trim();
15         if (msg.length() > 0) {
16             if (!handleCommand(msg, "Master")) {
17                 // Not a command, treat as regular chat message
18             }
19         }
20     }
21 };

```

Logic:

- [ServerCallbacks](#) : Handles connect/disconnect events
- [RxCallbacks](#) : Handles data received from Master

6.2.4 setup(): Initialization

```

1  void setup() {
2      Serial.begin(115200);
3      pinMode(LED_PIN, OUTPUT);
4
5      BLEDevice::init("ESP32_BLE_Slave");
6      BLEDevice::setMTU(512);
7
8      pServer = BLEDevice::createServer();
9      pServer->setCallbacks(new ServerCallbacks());
10
11     BLEService *pService = pServer->createService(SERVICE_UUID);
12
13     // TX characteristic (notify)
14     pTxCharacteristic = pService->createCharacteristic(
15         CHARACTERISTIC_UUID_TX, BLECharacteristic::PROPERTY_NOTIFY);
16     pTxCharacteristic->addDescriptor(new BLE2902());
17
18     // RX characteristic (write)
19     BLECharacteristic *pRxCharacteristic = pService->createCharacteristic(
20         CHARACTERISTIC_UUID_RX,
21         BLECharacteristic::PROPERTY_WRITE | BLECharacteristic::PROPERTY_WRITE_NR);
22     pRxCharacteristic->setCallbacks(new RxCallbacks());
23
24     pService->start();
25
26     BLEAdvertising *pAdvertising = BLEDevice::getAdvertising();
27     pAdvertising->addServiceUUID(SERVICE_UUID);
28     pAdvertising->setScanResponse(true);
29     pAdvertising->start();
30 }

```

Logic:

1. Initialize serial and LED
2. Initialize BLE device
3. Create server, register connection callbacks
4. Create GATT service and two characteristics
5. Start service and advertising

Key points:

- [pAdvertising->addServiceUUID\(SERVICE_UUID\)](#) : Includes service UUID in advertising, so Master can discover via UUID filtering

- `pAdvertising->setScanResponse(true)` : Enables scan response, allowing Master to get more device info

[Back to Elegoo](#)

6.2.5 loop(): Main Loop

```
1 void loop() {
2   if (Serial.available()) {
3     String msg = Serial.readStringUntil('\n');
4     msg.trim();
5     if (msg.length() > 0) {
6       if (!handleCommand(msg, "Local")) {
7         if (deviceConnected) {
8           sendViaBLE(msg);
9         } else {
10          Serial.println("Not connected...");
11        }
12      }
13    }
14  }
15  delay(10);
16 }
```

Logic: Checks serial input, first determines if it's a local command, otherwise sends to Master via BLE.

6.3 Master Code Walkthrough: ESP32_Master.ino

6.3.1 Configuration Section

```
1 BLEClient *pClient = nullptr;
2 BLEScan *pScan = nullptr;
3 BLERemoteCharacteristic *pTxCharacteristic = nullptr;
4 BLERemoteCharacteristic *pRxCharacteristic = nullptr;
5
6 bool deviceConnected = false;
7 bool shouldConnect = false;
8 bool scanDone = false;
9 BLEAdvertisedDevice *targetDevice = nullptr;
```

Key points:

- `pClient` : BLE client, used to connect to Slave
- `pTxCharacteristic` : Remote TX characteristic (receive data from Slave)
- `pRxCharacteristic` : Remote RX characteristic (send data to Slave)
- `scanDone` : Scan complete flag
- `targetDevice` : Stores scanned target device info

6.3.2 Scan Callback

```
1 class ScanCallbacks : public BLEAdvertisedDeviceCallbacks {
2   void onResult(BLEAdvertisedDevice advertisedDevice) {
3     if (advertisedDevice.haveServiceUUID() &&
4         advertisedDevice.getServiceUUID().toString() == SERVICE_UUID) {
5       targetDevice = new BLEAdvertisedDevice(advertisedDevice);
6       pScan->stop();
7       scanDone = true;
8     }
9   }
10 };
```

Logic: Triggered for each device found during scan, checks if service UUID matches.

Key points:

- `advertisedDevice.haveServiceUUID()` : Checks if advertising contains service UUID
- `pScan->stop()` : Stops scanning immediately after finding target
- `scanDone = true` : Notifies main loop to start connecting

6.3.3 Connect to Slave

```

1  bool connectToSlave(){
2      pClient = BLEDevice::createClient();
3      pClient->setClientCallbacks(new ClientCallbacks());
4
5      if (!pClient->connect(targetDevice)){
6          return false;
7      }
8
9      BLERemoteService *pService = pClient->getService(SERVICE_UUID);
10     if (pService == nullptr) {
11         pClient->disconnect();
12         return false;
13     }
14
15     pTxCharacteristic = pService->getCharacteristic(Characteristic_UUID_TX);
16     pRxCharacteristic = pService->getCharacteristic(Characteristic_UUID_RX);
17
18     if (pTxCharacteristic->canNotify()) {
19         pTxCharacteristic->registerForNotify(notifyCallback);
20     }
21
22     return true;
23 }

```

Logic:

1. Create BLE client
2. Connect to target device
3. Get GATT service
4. Get both characteristics
5. Subscribe to TX notifications

Key points:

- `pClient->connect()` : Initiates GATT connection
- `pClient->getService()` : Discovers service from Slave
- `registerForNotify()` : Registers notification callback to receive Slave data

6.3.4 Notification Callback

```

1  void notifyCallback(BLERemoteCharacteristic *pCharacteristic,
2      uint8_t *data, size_t length, bool isNotify) {
3      String msg = "";
4      for (size_t i = 0; i < length; i++) {
5          msg += (char)data[i];
6      }
7      msg.trim();
8      // Parse and display message
9  }

```

Logic: When Slave sends a notification, this callback is automatically invoked, converting byte data to a string and displaying it.

6.3.5 setup() and loop()

```

1  void setup() {
2      Serial.begin(115200);
3      BLEDevice::init("ESP32_BLE_Master");
4      startScan();
5  }
6
7  void loop() {
8      if (scanDone) {
9          connectToSlave();
10     }
11     if (shouldConnect && !deviceConnected) {
12         connectToSlave();
13     }
14     if (Serial.available()) {
15         // Read serial input, send to Slave
16     }
17 }

```

```
18 | delay(10);
    | }
```

[Back to Elegoo](#)

Logic:

1. `setup()` : Initialize BLE and start scanning
2. `loop()` : Check scan status, connection status, and serial input
3. Auto-reconnect after disconnection

6.4 Customization Guide

6.4.1 Change Bluetooth Name

Slave side:

```
1 | BLEDevice::init("Your_Slave_Name");
```

Master side:

```
1 | BLEDevice::init("Your_Master_Name");
```

6.4.2 Change LED Pin

Modify in both files:

```
1 | #define LED_PIN 2 // Change to your desired pin
```

Common ESP32 dev board onboard LED pins:

- ESP32 Dev Module: GPIO2
- ESP32-CAM: GPIO4
- Some boards: GPIO4 / GPIO13, etc.

6.4.3 Add New Commands

Add an `if` branch in the `handleCommand()` function in **both files**. New commands follow the "local input → forward to remote for execution, remote command → execute locally" pattern:

```
1 | // Write this in both Slave and Master's handleCommand()
2 | if (msg == "/blink"){           // Blink remote LED 3 times
3 |   if (source == "Local"){
4 |     sendViaBLE("/blink");       // Local input → forward command to remote
5 |     Serial.println(">>> Sent blink command.");
6 |   } else {                      // source == "remote name"
7 |     for (int i = 0; i < 3; i++){ // Remote command → execute locally
8 |       digitalWrite(LED_PIN, HIGH); delay(200);
9 |       digitalWrite(LED_PIN, LOW); delay(200);
10 |    }
11 |  }
12 |  return true;
13 | }
```

 **Both sides must be modified in sync:** Since commands are handled on both sides independently, new commands must be added to both Slave and Master code.

 **source parameter rules:** On the Master side, "**Local**" = local serial input, "**Slave**" = notification from Slave; on the Slave side, "**Local**" = local serial input, "**Master**" = write from Master.

6.4.4 Change Service UUID

To customize UUIDs (must be identical in both files):

```
1 | #define SERVICE_UUID "your-uuid-here"
2 | #define CHARACTERISTIC_UUID_RX "your-uuid-here"
3 |
```

#define CHARACTERISTIC_UUID_TX "your-uuid-here"

[Back to Elegoo](#)



💡 You can use an online UUID generator (e.g., <https://www.uuidgenerator.net/>) to generate random UUIDs.

6.4.5 Change MTU Size

1 | `BLEDevice::setMTU(256); // Min 23, max ~517`



⚠ MTU settings on both sides don't need to be identical — the actual MTU is the smaller of the two. The default 512 is sufficient for chat.

[← Bluetooth web](#)

[over view→](#)